From Flow Matching to Mean Flow

> This post introduces flow matching and mean flow for image generation, aiming to provide a thorough and detailed introduction to these two flow models.
>
> Prerequests are **Linear Algebra**, **Probability Theory** and **Multivariable Calculus**, and a very little bit of **Deep Learning**.

1 Basics for Image Generation

> Abstract:
>
> We consider the task of generating images as generating objects that are represented as **vectors** $z \in \mathbb{R}^d$. Generation is the task of **sampling from a probability distribution** p_{data}, and we have access to a dataset of samples $z_1, \dots, z_N$ from p_{data} during training. Conditional generation assumes that we condition the distribution on a label y, and we want to sample from the conditional distribution $p_{\text{data}}(\cdot \mid y)$ that having access to data set of pairs $(z_1, y), \dots, (z_N, y)$ during training. Our goal is to train a generative model to transform samples from a simple distribution p_{init} (e.g. a Gaussian Distribution) into samples from p_{data}, the target distribution.

1.1 Images as Vectors

Let's begin with representation of images that we will encounter, as well as how we will go about representing them numerically.

Consider a gray image of $h \times w$ pixels, where h and w denote the **height** and **width** of the image. Then, a gray image can be represented as a **matrix**:

$$\text{Gray-Image} = \begin{bmatrix} x_{11} & x_{12} & \dots & x_{1w} \\ x_{21} & x_{22} & \dots & x_{2w} \\ \vdots & \vdots & \ddots & \vdots \\ x_{h1} & x_{h2} & \dots & x_{hw} \end{bmatrix}_{h \times w}$$

where each pixel is assigned an intensity value $x_{ij} \in \mathbb{R}$, and $\text{Gray-Image} \in \mathbb{R}^{h \times w}$.

Then expand the **Gray images** into $h \times w$ **RGB images** with three color channels, where each channel is assigned with an intensity matrix as the gray images. That is:

$$\text{RGB-Image} = \begin{bmatrix} \begin{bmatrix} r_{11} & \cdots & r_{1w} \\ \vdots & \ddots & \vdots \\ r_{h1} & \cdots & r_{hw} \end{bmatrix} \\ \begin{bmatrix} g_{11} & \cdots & g_{1w} \\ \vdots & \ddots & \vdots \\ g_{h1} & \cdots & g_{hw} \end{bmatrix} \\ \begin{bmatrix} b_{11} & \cdots & b_{1h} \\ \vdots & \ddots & \vdots \\ b_{h1} & \cdots & b_{hw} \end{bmatrix} \end{bmatrix}_{3\times h\times w}$$

To combine both **gray image** and **rgb images**, we represent an image:

$$\text{Image} \in \mathbb{R}^{c\times h\times w},$$

where c, h, w represent the number of color channels, image height and width respectively. ($\text{Image} \in \mathbb{R}^{h\times w\times c}$ is also acceptable, but with another interpretation: matrix whose elements are vectors of $\mathbb{R}^c$)

For any $\text{Image} \in \mathbb{R}^{c\times h\times w}$, we can flatten them into a vector $z \in \mathbb{R}^d$, where $d = c\cdot h\cdot w$.

For example:

$$\text{RGB-Image} = \begin{bmatrix} \begin{bmatrix} r_{11} & \cdots & r_{1w} \\ \vdots & \ddots & \vdots \\ r_{h1} & \cdots & r_{hw} \end{bmatrix} \\ \begin{bmatrix} g_{11} & \cdots & g_{1w} \\ \vdots & \ddots & \vdots \\ g_{h1} & \cdots & g_{hw} \end{bmatrix} \\ \begin{bmatrix} b_{11} & \cdots & b_{1h} \\ \vdots & \ddots & \vdots \\ b_{h1} & \cdots & b_{hw} \end{bmatrix} \end{bmatrix}_{3\times h\times w} \Rightarrow \begin{bmatrix} r_{11} \\ \vdots \\ r_{1w} \\ \vdots \\ r_{hw} \\ \vdots \\ g_{11} \\ \vdots \\ g_{hw} \\ \vdots \\ b_{11} \\ \vdots \\ b_{hw} \end{bmatrix}_{3\cdot h\cdot w} \in \mathbb{R}^{3\cdot h\cdot w}.$$

Therefore, throughout this post,

$$\text{we identify images to be generated as vector:} \quad z \in \mathbb{R}^d.$$

1.2 Sampling as Generation

> For simplicity, throughout this post, we don't explicitly distinguish $\text{Random Variable } Z$ and sample z. When writing $z/Z \sim \text{Distribution}$, it means:
>
> z/Z is either a sample sampled from Distribution, or a random variable that follows the Distribution.

Let's define what's means to **"generate iamge"**.

For example, let's say we want to generate an image of a dog. Naturally, there are many possible images of dogs that we would be happy with. In particular, there's no one single "best" image of a dog. Rather, there's **a spectrum of images** that fit better or worse. In machine learning, it is common to think of this diversity of possible images as **a probability distribution**. We call it the **data distribution** and denote it as p_{data}. In the example of dog images, this distribution would therefore give higher likelihood to images that look more like a dog. Therefore, how "good" an image fits - a rather subjective statement - is replaced by how "likely" it is under the data distribution p_{data}. With this, **we can mathematically express the task of generation as sampling from the (unknown) distribution** p_{data}.

$$\text{Generating an object } z \text{ is modeled as sampling from the data distribution } p_{\text{data}}.$$

A **generative model** is a machine learning model that allows us to generate samples from p_{data}. In machine learning, we require data to train models. In generative modeling, we usually assume access to a finite number of examples (train data) sampled independently from p_{data}, which together serve as a proxy for the true distribution.

$$\text{A dataset consists of a finite number of samples} \quad z_1, z_2 \dots z_N \sim p_{\text{data}}.$$

As the size of our dataset grows very large, it becomes an increasingly better representation of the underlying distribution p_{data}.

1.3 Conditional Generation

> The notation of conditional disctibution $p(\cdot \mid y)$ describe a distribution p conditions on y. Here, "$\cdot$" means you can insert any position x at "$\cdot$" to obtain the probability densitity $p(x \mid y)$.
>
> Throughout this post, we don't explicitly distinguish between the probability density and the probability distribution. The notation p means either a probability distribution or a probability density.

In many cases, we want to generate an object conditioned on some data y. For example, we might want to generate an image conditioned on $y =$ "a dog running down a hill covered with snow with mountains in the background". We can rephrase this as sampling from a conditional distribution:

$$\text{Conditional generation involves sampling from } z \sim p_{\text{data}}(\cdot \mid y), \text{ where } y \text{ is a conditioning variable.}$$

We call $p_{\text{data}}(\cdot \mid y)$ the **conditional data distribution**. The conditional generative modeling task typically involves **learning to condition on an arbitrary, rather than fixed, choice of** y. Using our previous example, we might alternatively want to condition on a different text prompt, such as $y =$ "a photorealistic image of a cat blowing out birthday candles". We therefore seek a single model which may be conditioned on any such choice of y. It turns out that techniques for unconditional generation are readily generalized to the conditional case. We will first focus on unconditional generation, but keep in mind that conditional generation is what we're building towards.

1.4 From Noise to Data

So far, we have discussed the what of generative modeling: generating samples from p_{data}. Here, we will briefly discuss the how.

For this, we assume that we have access to some **initial distribution** p_{init} that we can easily sample from, such as the **Gaussian Distribution** $p_{\text{init}} = N(0, I_d)$. The goal of generative modeling is then to **transform samples from** $x \sim p_{\text{init}}$ into samples from p_{data}. We note that p_{init} does not have to be so simple as a **Gaussian**. As we shall see, there are interesting use cases for leveraging this flexibility. Despite this, in the majority of applications we take it to be a **simple Gaussian** and it is important to keep that in mind.

---

2 Flow Basics: the Continuity Equation

> This section is not necessary to understand the flow matching model, but it will offer some insights into a deeper understanding of the background of flow matching.
>
> It presents a detailed walkthrough of the **continuity equation**, from physical intuition to mathematical formalism. We aim to derive the equation step by step and explain all quantities involved.

2.1 Motivation: Conservation in Physical Systems

Many physical quantities are conserved in nature. For instance:

- **Mass** in a closed system
- **Charge** in electromagnetic systems
- **Energy** in isolated systems
- **Number of particles** in fluid dynamics

Although these quantities are conserved **globally**, they **vary locally** in both space and time. To describe their local behavior, we define **density functions** that describe how much of a given conserved quantity exists per unit volume at any point and time:

- Mass density: $\rho(x, t)$
- Charge density: $\rho_q(x, t)$
- Energy density: $\epsilon(x, t)$
- Particle number density: $n(x, t)$

These are all functions $\mathbb{R}^n \times \mathbb{R}^+ \to \mathbb{R}$ (typically, $n = 3$).

To model how these densities evolve, we introduce the idea of an abstract **fluid parcel** at position x and time t. This fluid parcel carries a certain amount of the conserved quantity. The idea of fluid parcel is purely conceptual, like a massless particle that moves with the flow, carrying the density value $\rho(x, t)$. Each parcel has a velocity $\mathbf{v}(x, t)$.

The goal for this section is to derive a **mathematical law** that describes how the distribution of this quantity evolves over space and time. This law is known as the **continuity equation**.

2.2 Fundamental Definitions

Let's define:

- $\rho(x,t) : \mathbb{R}^n \times \mathbb{R}^+ \to \mathbb{R}$, the scalar **density function** of a conserved quantity
- $\mathbf{v}(x,t) : \mathbb{R}^n \times \mathbb{R}^+ \to \mathbb{R}^n$, the velocity of the moving fluid parcel
- $\mathbf{J}(x,t) : \mathbb{R}^n \times \mathbb{R}^+ \to \mathbb{R}^n$, the **flux vector field**, giving how much quantity passes through a unit area per unit time

For a conserved quantity transported by the velocity field $\mathbf{v}(x,t)$, the **flux** is given by:

$$\mathbf{J}(x,t) \triangleq \rho(x,t) \cdot \mathbf{v}(x,t)$$

Now consider a fixed spatial region $V \subset \mathbb{R}^n$ with boundary ∂V:

- The **total quantity inside V at time t** is:

$$Q_V(t) = \int_V \rho(x,t)\, dx$$

- The **time derivative** of $Q_V(t)$ represents how the total amount is changing of the spatial region V:

$$\frac{d}{dt} Q_V(t) = \frac{d}{dt} \int_V \rho(x,t)\, dx$$

This change must result from **flux across the boundary** ∂V.

$$\begin{cases} \frac{d}{dt}\int_V \rho(x,t)\, dx > 0 : & \text{Total quantity inside } V \text{ increases} \\ \frac{d}{dt}\int_V \rho(x,t)\, dx < 0 : & \text{Total quantity inside } V \text{ decreases} \end{cases}$$

2.3 (Optional) Flux Across the Boundary

For each point $x \in \partial V$, we define the **outward unit normal vector** n_x . For $dS_x,$ an **infinitesimal surface element** at x , define the infinitesimal oriented surface vector element as $d\mathbf{S_x} = \mathbf{n}_x\, dS_x$, pointing outward.

Then the **net outflow** of quantity through ∂V is:

$$\oint_{\partial V} \mathbf{J}(x,t)\, d\mathbf{S_x}$$

This measures how much of the quantity is **leaving** the region per unit time.

$$\begin{cases} \oint_{\partial V} \mathbf{J}(x,t)\, d\mathbf{S_x} > 0 : & \text{flux exits the region } \partial V \\ \oint_{\partial V} \mathbf{J}(x,t)\, d\mathbf{S_X} < 0 : & \text{flux enters the region } \partial V \end{cases}$$

Naturally, by the conservation of the quantity, we obtain:

$$\frac{d}{dt}\int_V \rho(x,t)\, dx = -\oint_{\partial V} \mathbf{J}(x,t)\, d\mathbf{S_x}$$

2.4 (Optional) Divergence

> Additional mathematical theory to understand the continuity equation. Feel free to skip this part if you grasp the divergence and Gauss's Divergence Theorem.

2.4.1 Definition and Geometric Interpretation

> Distinguish the divergence $\nabla \cdot \mathbf{J}$ and the gradient $\nabla \mathbf{J}$.

The divergence of a vector field is defined as:

$$\nabla_x \cdot \mathbf{J}(x,t) \triangleq \lim_{V \to 0} \oint_{\partial V} \mathbf{J}(x,t) d\mathbf{S}_\mathbf{x}$$

- If $\nabla_x \cdot \mathbf{J}(x,t) > 0$, more is **flowing out** than in $\Rightarrow$ local decrease in ρ, and the position x is a **source**.
- If $\nabla_x \cdot \mathbf{J}(x,t) < 0$, more is **flowing in** $\Rightarrow$ local increase in ρ, and the position x is a **sink**.
- If $\nabla_x \cdot \mathbf{J}(x,t) = 0$, net flow is balanced $\Rightarrow$ ρ stays unchanged

This is a **local measure** of how much the vector field is **“spreading out”** at a point.

This geometric interpretation is essential: **divergence at a point tells you whether the field is expanding (positive) or contracting (negative) at that location.**

Analytically, the previous definition is equal to:

$$\lim_{V \to 0} \oint_{\partial V} \mathbf{J}(x,t) d\mathbf{S}_\mathbf{x} = \sum_{i=1}^{n} \frac{\partial J_i(x,t)}{\partial x_i}$$

The proof of the equation $\lim_{V \to 0} \oint_{\partial V} \mathbf{J}(x,t) d\mathbf{S}_\mathbf{x} = \sum_{i=1}^{n} \frac{\partial J_i(x,t)}{\partial x_i}$ is in **Appendix, I. Geometric and Analytical Definition of Divergence**.

2.4.2 Gauss’s Divergence Theorem

> The rigorous proof is ommited. We only provide an intuitive interpretation of the **Gauss's Divergence Theorem**.

$$\oint_{\partial V} \mathbf{J}(x,t)\, d\mathbf{S}_x = \int_V \nabla_x \cdot \mathbf{J}(x,t)\, dx$$

The theorem relates the total outward flux through the boundary to the **divergence** of $\mathbf{J}$ inside the volume. Geometrically:

- The left-hand side sums the quantity flowing **out of** every surface patch
- The right-hand side sums the **local expansion or compression** of flow inside V

This formula transforms the boundary integral into a volume integral.

2.5 Continuity Equation

Utilize the **Gauss's Divergence Theorem**:

$$\frac{d}{dt}\int_V \rho(x,t)\,dx = -\oint_{\partial V} \mathbf{J}(x,t)\,d\mathbf{S}_\mathbf{x}$$

$$= -\int_V \nabla_x \cdot \mathbf{J}(x,t)\,dx$$

Suppose ρ is smooth enough, we have:

$$\frac{d}{dt}\int_V \rho(\mathbf{x},t)\,d\mathbf{x} = \lim_{\Delta t\to 0}\frac{\int_V \rho(\mathbf{x},t+\Delta t)\,d\mathbf{x} - \int_V \rho(\mathbf{x},t)\,d\mathbf{x}}{\Delta t}$$

$$= \lim_{\Delta t\to 0}\int_V \frac{\rho(\mathbf{x},t+\Delta t)-\rho(\mathbf{x},t)}{\Delta t}\,d\mathbf{x}$$

$$= \int_V \lim_{\Delta t\to 0}\frac{\rho(\mathbf{x},t+\Delta t)-\rho(\mathbf{x},t)}{\Delta t}\,d\mathbf{x}$$

$$= \int_V \frac{\partial \rho(\mathbf{x},t)}{\partial t}\,d\mathbf{x}$$

Thus:

$$\int_V \frac{\partial \rho(x,t)}{\partial t}\,dx = -\int_V \nabla_x \cdot \mathbf{J}(x,t)\,dx$$

$$\int_V \left(\frac{\partial \rho(x,t)}{\partial t} + \nabla_x \cdot \mathbf{J}(x,t)\right)dx = 0$$

$$\frac{\partial \rho(x,t)}{\partial t} + \nabla_x \cdot \mathbf{J}(x,t) = 0$$

Finally, using $\mathbf{J}(x,t) = \rho(x,t)\cdot \mathbf{v}(x,t)$, we obtain the continuity equation which expresses **local conservation** of the quantity carried by the flow.

$$\text{Continuity Equation: } \frac{\partial \rho(x,t)}{\partial t} + \nabla_x \cdot (\rho(x,t)\cdot \mathbf{v}(x,t)) = 0$$

The **continuity equation** arises naturally when we model **a conserved quantity flowing through space**. It reflects a deep idea:

> Any local change in the quantity must be explained by its flow into or out of that region.

This continuity equation is fundamental in physics and mathematics, appearing in fluid mechanics, electromagnetism, quantum mechanics, and beyond.

---

3 Flow Matching Model

> Abstracts:

> We first introduce the tranditional flow models and its disadvantages. Then, **two perspectives** of **stochastic process** are provided, explaining why the **velocity vector field** $v(x,t)$ can fully transoform a sample from p_{init} to a sample from p_{data}. How we model the stochastic process with the **continuity equation**. Since $v(x,t)$ can fully transform samples from p_{init} to p_{data}, what we need to do is simply modeling this **velocity vector field** with **Neural Network**. In the third section, we talk about how to constructing a training target, **the marginal velocity**, and in the fourth section, we prove how the **marginal velocity** collapese into a simpler **conditional velocity**. After demonstrating a thorough process of training the model, we shift our focus to **conditional generation**, as well as **CFG**.

3.1 Flow Models: Transform of Random Variables

For two distributions: $p_{\text{init}}, p_{\text{target}}$, and two random vectors: $x \sim p_{\text{init}}, y \sim p_{\text{target}}$, suppose we know the **pdf** of $p_{\text{init}}(x)$, and their transform relationships $y = g(x)$.

Then, for any sample of p_{init}, we can transform it into samples of p_{target} by leveraging the **transforms of random variables**:

$$\left| \int p_{\text{init}}(x)dx \right| = \left| \int p_{\text{target}}(y)dy \right| \Rightarrow p_{\text{target}}(y) = p_{\text{init}}(x) \left| \frac{\partial x}{\partial y} \right|,$$

where $\left| \frac{\partial x}{\partial y} \right|$ denotes the absolute value of the determinant of the **Jacobian Matrix** of function $x = g^{-1}(y)$.

For image generation, the p_{target} is exactly p_{data} mentioned above, and thus, we succeed in sampling objects from p_{data} (generating images).

However, the flow model are practically difficult for the following the following reason:

- The time complexity of calculating determinant is $O(n^3)$, and here, when $n = 3 \times 256 \times 256$, thus the exponent part will largely increase the computation costs.

Then, came the **flow matching** method!

3.2 Stochastic Process and "Flows"

> This part utilizes the concept of **Stochastic Process** and **II. Flow Basics: the Continuity Equation (Optional)** to model the transforms of random variables.

Transform directly from p_{init} to p_{data} is difficult. So, this time, we insert the intermediate distributions. That is, instead of directly transforming $p_{\text{init}} \Rightarrow p_{\text{data}}$, we do: $p_{\text{init}} \Rightarrow p_{\text{intermediate 1}} \Rightarrow p_{\text{intermediate 2}} \Rightarrow \ldots \Rightarrow p_{\text{data}}$. To model these intermediate processes, we introduce another variable $t \in [0,1]$, and define the distribution at time t as p_t. Specifically, when $t = 0$, $p_0 = p_{\text{init}}$, and when $t = 1, p_1 = p_{\text{data}}$.

3.2.1 Model Transforms as Stochastic Process

> A stochastic process is defined as $X : \mathcal{F} \times [0,1] \to \mathbb{R}^d, (A,t) \to X(A,t)$, where A is the **event** and t is **time**.

- **Perspective I: A Collection of Random Variables**

> Here, X_t and $X_t(A)$ all represent the random variable, with the second one explicitly writes the event A.

For any fixed time t, the stochastic process collapses into a **random variable**. i.e., $X_t : \mathcal{F} \to \mathbb{R}^d, A \to X_t(A)$, and $X_t \sim p_t$.

For example, $X_0 \sim p_0 = p_{\text{init}}$, and $X_1 \sim p_1 = p_{\text{data}}$. As time t evolves from 0 to 1, we in sequence get a collection of random variables, which represents how random variables evolve as time goes by.

From this perspective, if we go through every $t \in [0, 1]$, then a stochastic process can be considered as **a collection of random vectors**. This perspective aligns with the flow model: **transforms of random variables**.

- **Perspective II: A Collection of Trajectories**

  For any fixed event A, the stochastic process collapses into a **trajectory**. i.e., $X_A : [0, 1] \to \mathbb{R}^d, t \to X_A(t)$.

  And, **a single trajectory represents the different mapping results of the event A under the random variables X_t when t evolving from 0 to 1.**

  As we go through evey $A \in \mathcal{F}$, we obtain **a collection of trajectories**.

  For $\forall A \in \mathcal{F}$, if we take the derivative of $X_A(t)$, a collection of trajectories, the result $\frac{d}{dt}X_A(t)$ is the corresponding **velocity !** Now, simply ommit the event A and focus solely on **trajectories**. We can define a **velocity vector field**:

  $$v : \mathbb{R}^d \times [0, 1] \to \mathbb{R}^d, (x, t) \to v(x, t),$$

  by taking the derivative of trajectories:

  $$\text{For } \forall A \in \mathcal{F}, \forall t \in [0, 1], \quad v\left(X_A(t), t\right) \triangleq \frac{d}{dt}X_A(t).$$

  > Note that the vector field v is simply a mapping from $\mathbb{R}^d \times [0, 1]$ to $\mathbb{R}^d$. That means, we don' t care from which trajectory the position x in $v(x, t)$ comes from. x is simply a point in $\mathbb{R}^d$. This definition also requires that, for different event A_1 and A_2, $X_{A_1}(t) \neq X_{A_2}(t)$ for $\forall t$. Or otherwise, if $X_{A_1}(t_i) = X_{A_2}(t_i)$ at t_i, then $X_{A_1}(t) = X_{A_2}(t)$ for $\forall t \in [t_i, 1]$.
  >
  > That doesn't mean two trajectories will never interact with each other. This simply means, for 2 different event A_1 and A_2, the mapping results at the same time t, i.e., under the transform of the same random variable X_t, should be different.

  From this perspective, if we sample x_0 from the initial distribution p_0, we can equivalently obtain a sample x_t from the distribution p_t leveraging the **velocity vector field**:

  $$x_t - x_0 = \int_0^t v(x, t)\, dt.$$

  When $t = 1$, we succeed to sample x_1 from p_{data}.

  Thus, equivalent to the transform of random variables in Perspective I, the velocity vector field in Perspective II can also fully achieve the goal of sampling from the target distribution p_{data}. If we sample every points from p_{init}, we equivalently sample every points from p_{data}. For simplicity, in the following post, we simply say **the velocity vector field transforms the p_{init} into p_{data}**, instead saying transforms a sample from p_{init} inot a sample from p_{data}. Compared to utilizing jacobians and determinants, this **ODE** is much easier to calculate. Up to now, the key idea is to **find a velocity vector field** that fully describes this stochastic process.

  If there has been a velocity, we do the following procedure to "sample from p_{data}".

  ```
  # Generation/Sample Process
  Requires:
  - initial distributin: p_init,
  - velocity: v(x, t)
  ```

```
set t = 0
set time step n
set step size h = 1 / n
sample x_0 from p_init
set x = x_0

# euler method to simulate the interal
for i in {0, 1, ..., n-1}, do
    x = x + h * v(x, i * h)
end for

return x
```

3.2.2 View Stochastic Process with "Flows"

> In the following post, when we say "flow conditions", it means there's a quantity that is conserved globally, but varies in both space and time. Or, be more specifically, at any time t, the quantity $Q_t = \int_x \rho(x,t)dx$ is conserved.

From **II Flow Basics**, we learn that the concepts of **"Flow"** and **the continuity equation** apply to the situations where a quantity is conserved globally at any time, but varies in both space and time.

In our case, the stochastic process, the probability mass $\int p_t(x)dx$ at any time $t \in [0,1]$ is always equal to one, and the probability density $p_t(x)$ varies over the time t and the space x. That means, the conserved quantity **probability mass** and the **probability density** $p_t(x)$ varying over time t and position x exactly satisfy the "flow conditions" we mentioned above!

Suppose there exists a stochastic process that transforms from the initial distribution to the target distribution, the velocity vector field $v(x,t)$ defined in the Stochastic Process, Perspective II and the corresponding probability density $p_t(x)$ must satisfy the continuity equation.

3.3 Constructing the Training Target

In previous sections, we've shown how $v(x,t)$ transform p_{init} into p_{data}. If we find $v(x,t)$, then all is done. In deep learning, we use a neural network $v^\theta(x,t)$ to approximate $v(x,t)$. We need to find a **training target** $v^{\text{tgt}}(x,t)$ and a **loss function** $\mathcal{L}(v^\theta, v^{\text{tgt}})$. In this section, we will propose a training target $v^{\text{tgt}}(x,t)$.

> Note that the difference between the $v(x,t)$ and $v^{\text{tgt}}(x,t)$. Here, $v(x,t)$ is the real but unknown velocity. We need to construct an acceptable velocity as the training target, and that is $v^{\text{tgt}}(x,t)$. $v(x,t)$ and $v^{\text{tgt}}(x,t)$ are not necessarily the same, but $v^{\text{tgt}}(x,t)$ must satisfy the role of $v(x,t)$, or in other words, $v^{\text{tgt}}(x,t)$ must transform the initial distribution p_{init} into the target distribution p_{data}, while satisfying the "flow conditions".

It's difficult to find an analytical $v(x,t)$ directly for arbitary (x,t) along the stochastic process from p_{init} to p_{data} . Thus, we look at a "conditional version" at first, and then use the conditional version to construct the marginal version.

3.3.1 Conditional and Marginal Probability Path

> The concept of **probability path** is similar to the conecpt of **the stochastic process**. A probability path describes how a stochasitc process evolves with time.

The first step of constructing the training target v^{tgt} is by specifying a probability path. Intuitively, a probability path specifies a gradual interpolation between the noise distribution p_{init} and the data distribution p_{data}. You can think of a probability path as a trajectory in the space of distributions.

A deterministic evolution trajectory from p_{init} to p_{data} is considered as a probability path. $p_0 = p_{\text{init}}, p_1 = p_{\text{data}}$, and the probability distribution at time $t \in [0, 1]$ is p_t. Since we will then introduce another kind of probability path, we call the probability path from p_{init} to p_{data} the **marginal probability path** .

For a data point $z \in \mathbb{R}^d$ from p_{data}, δ_z denotes the simplest distribution: **sampling from δ_z always returns z** (i.e., δ_z is deterministic). A **conditional (interpolating) probability path** is a set of distribution $p_t(\cdot \mid z)$ over $\mathbb{R}^d$ such that:

$$p_0(\cdot \mid z) = p_{\text{init}}, \text{ and } p_1(\cdot \mid z) = \delta_z, \quad \text{for all } z \in \mathbb{R}^d.$$

In other words, a conditional probability path gradually converts the initial noisy distribution p_{init} into a single data point z.

The densitity relationship between the conditional and the marginal probability path is given by:

$$p_t(x) = \int p_t(x \mid z) p_{\text{data}}(z) dz$$

Here, $p_t(x)$ means the probability density to sample x from the distribution p_t, $p_t(x \mid z)$ means the probability desity to sample x from the distribution $p_t(\cdot \mid z)$, and $p_{\text{data}}(z)$ means the probability density to sample z from the distribution p_{data}. Utilizing the **total probability theorem**, it's easy to gain the result that $p_t(x) = \int p_t(x \mid z) p_{\text{data}}(z) dz$.

3.3.2 Conditional and Marginal Velocity

3.3.2.1 Conditional Velocity

We firstly define the conditional velocity.

For a conditional probability path,

$$X_t(\cdot \mid z) = \alpha_t z + \beta_t X_0,$$

where $X_t(\cdot \mid z) \sim p_t(\cdot \mid z)$, and $X_0 \sim p_0(\cdot \mid z) = p_{\text{init}}, X_1 \sim p_1(\cdot \mid z) = \delta_z$ for all $z \sim p_{\text{data}}$.

Here, α_t and β_t are two simple monotonic functions of t: $[0, 1] \to [0, 1]$, and:

$$\begin{cases} \alpha_0 = 0 \\ \\ \beta_0 = 1 \end{cases} \quad \text{and} \quad \begin{cases} \alpha_1 = 1 \\ \\ \beta_1 = 0 \end{cases}$$

α_t and β_t are simple functions like:

$$\begin{cases} \alpha_t = t \\ \\ \beta_t = 1 - t \end{cases} \quad \text{or} \quad \begin{cases} \alpha_t = \sqrt{t} \\ \\ \beta_t = \sqrt{1 - t} \end{cases}$$

The conditional velovity can be written as:

$$\text{For all } \omega \in \Omega, \text{ all } z \in \mathbb{R}^d, \text{ at position } x = X_t(\omega \mid z), \quad v(x, t \mid z) \triangleq \frac{d}{dt} X_t(\omega \mid z) = \dot{\alpha}_t z + \dot{\beta}_t X_0.$$

> The reason we use $x = X_t(w \mid z)$ is that x simply implies a position. Note that, for a condition z, the input of the conditional velocity is the position x and the time t. And on the right hand, $\frac{d}{dt} X_t(\omega \mid z)$, it implies that $X_t(\omega \mid z)$ is a function of time t.

The velocity $v(x, t \mid z)$ means :**the velocity of a trajectory whose destination is z at position x and time t**. The **condition velocity** means **this velocity is conditioned that the destination of the trajectory is z**.

3.3.2.2Marginal Velocity

We've show how to construct a **conditional velocity**, i.e., at position x and time t, condition on the fact that the destination is z, the corresponding velocity is $v(x, t \mid z)$. But, what we need is the **marginal velocity** $v(x, t)$, that is the velocity at position x and time t. A natural way is to construct the marginal velocity based on the conditional velocity, or more specifically, take the weighted average of the conditional velocity to get the marginal velocity. The marginal velocity at position x and time t equals to the weighted average of the conditional velocity. Here, $v(x, t \mid z)$ is the velocity at (x, t) conditioned on the fact that the destination is z. $p_t(z \mid x)$ means that **at time t and position x, the probability density that the final destination is z**. Utilizing the **Bayes Formula**, we get the final results:

$$\begin{aligned} v^{\text{tgt}}(x, t) &= \int v(x, t \mid z) p_t(z \mid x) \, dz \\ &= \int v(x, t \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} \, dz \end{aligned}$$

It's a little tricky that $p_t(z \mid x)$ means **at (x, t), the probability density that the destination is z**, and it seems more confusing that $p_t(z \mid x) = \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}$. It's hard to **proove** this statement, but here's an explanation to help you understand it. $p_t(x \mid z)$ is the probability density to sample x from the distribution $p_t(\cdot \mid z)$. $p_t(\cdot \mid z)$ also implies that the terminal point of this conditional probaility path is z, and the $p_t(\cdot \mid z)$ is the distribution at time t. This means, another interpretation of $p_t(x \mid z)$ is, at time t and position x, the probability density that the terminal point is z is $p_t(x \mid z)$. Similiarily, $p_t(x)$, which initially means the probability denstiy to sample x from p_t, implies $p_t(x)$ is the probabiliy density that at time t and position x, the probability density that the denstination is z is $p_t(x)$. Based on these two interpretations, it's more acceptable that we can use the **Bayes Formula** to obtain that $p_t(x) = \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)}$, and $p_t(x \mid z)$ means the probability density that at time t and position x, the final position is z.

> This confusion arises from the property of the probability theorem and the notation of p_t. Anyway, in my perspective, the probability theorem is science, instead of mathematics. It's just a mathematical model we construct to explain the *plausibility*. From the perspective of function, we should simply write $p_t(x \mid z)$ as $p(x, t, z)$, since x, t, z are simply three input variables and the probability density is simply the mapping results. But, we choose to write it as $p_t(x \mid z)$, where we choose to use *the distribution at time* t to model the variable t, and the "conditional probability" to model the variable z. This is simply how we choose to model, and that' s absolutely not rigorous.

3.3.3 (Optional) Checking "Flow Conditions"

In the conditional probability path, since we simply scale and shift the random variable $X_0 \sim p_0$, it must follow the "flow condition": $\int p_t(x \mid z)\, dx = \text{constant}$, and the conditional velocity $v(x, t \mid z)$ can successfully transfrom the distribution $p_0(\cdot \mid z)$ into $p_1(\cdot \mid z)$. In a word, the probability density and the corresponding velocity follow the continuity equation $\frac{\partial p_t(x \mid z)}{\partial t} + \nabla_x \cdot (p_t(x \mid z) \cdot v(x, t \mid z))$.

But, does the constructed $v^{\text{tgt}}(x, t) = \int v(x, t \mid z) p_t(z \mid x)\, dz$ can successfully drive the initial distribution p_{init} into the target distribution p_{data}, and does this transform satisfies the "flow conditions"? Anyway, v^{tgt} is just the velocity we construct, and we don't know whether the probability mass $\int p_t(x \mid z) dz$ is conserved. To check that our construction is valid, we utilize the continuity equation:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x) &= \frac{\partial}{\partial t} \int p_t(x \mid z) p_{\text{data}}(z) dz \\ &= \int \frac{\partial}{\partial t} p_t(x \mid z) p_{\text{data}}(z) dz\end{aligned}$$

Then, use the continuity equation of the conditional probability path:

$$\frac{\partial}{\partial} p_t(x \mid z) + \nabla_x \cdot (p_t(x, \mid z) \cdot v(x, t \mid z)) = 0$$

and substitude the $\frac{\partial}{\partial t} p_t(x \mid z)$:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x) &= \int -\nabla_x \left(p_t(x \mid z) \cdot v(x, t \mid z)\right) p_{\text{dat}}(z)\, dz \\ &= -\int \nabla_x \cdot (p_t(x \mid z) \cdot v(x, t \mid z) p_{\text{data}}(z)) dz \\ &= -\nabla_x \cdot \int v(x, t \mid z) \cdot p_t(x \mid z) p_{\text{data}}\, dz \\ &= -\nabla_x \cdot \int v(x, t \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} p_t(x) dz \\ &= -\nabla_x \cdot \left(\left(\int v(x, t \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} dz \right) \cdot p_t(x) \right)\end{aligned}$$

Then, according to the definintion of $v^{\text{tgt}}(x, t) \triangleq \int v(x, t \mid z) p_t(z \mid x)\, dz = \int v(x, t \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} dz$:

$$\begin{aligned}\frac{\partial}{\partial t} p_t(x) &= -\nabla_x \cdot \left(v^{\text{tgt}}(x, t) \cdot p_t(x)\right) \\ \frac{\partial}{\partial t} p_t(x) + \nabla_x \cdot (v^{\text{tgt}}(x, t) \cdot p_t(x)) &= 0\end{aligned}$$

Amazing! The constructed $v^{\text{tgt}}(x, t)$ satisfies the continuity equation, and thus the "flow conditions" are guaranteed.

3.4 Training the Model

3.4.1 Substitute $v^{\text{tgt}}(x, t)$ with $v(x, t \mid z)$

In the previous section, we've showed how to construct the training target $v^{\text{tgt}}(x, t) = \int v(x, t \mid z) \frac{p_t(x \mid z) p_{\text{data}}(z)}{p_t(x)} dz$. However, there's no closed-form solution of the training target v^{tgt}, since we have no access to the distribution $p_t(x)$ and $p_{\text{data}}(z)$. What we have is only the initial distribution p_{init} and finite samples from p_{data} (the datasets). Thus, we have no access to our training target, $v^{\text{tgt}}(x, t)$. In this section, we will show that, for a specific kind of loss function: mean square loss, we can use the conditional velocity to substitute the marginal velocity and get the same training object.

The MSE loss of the flow matching model is given by:

$$\begin{aligned}\mathcal{L}_{\mathrm{FM}}(\theta) &= \mathbb{E}_{t,x\sim p_t}\left[\left\|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\right\|_2^2\right] \\ &= \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\right\|_2^2\right]\end{aligned}$$

And, the conditional MSE loss is given by:

$$\mathcal{L}_{\mathrm{CFM}}(\theta) = \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)-v(x,t\mid z)\right\|_2^2\right]$$

Since the loss is a function of θ, x and t are all considered as constants independent of θ. In this section, we aim to show that:

$$\mathcal{L}_{\mathrm{FM}}(\theta) = \mathcal{L}_{\mathrm{CFM}}(\theta) + C$$

Or, in other words, take the gradient form:

$$\nabla_{\theta}\mathcal{L}_{\mathrm{FM}}(\theta) = \nabla_{\theta}\mathcal{L}_{\mathrm{CFM}}(\theta)$$

Here is the proof:

$$\begin{aligned}\mathcal{L}_{\mathrm{FM}}(\theta) &= \mathbb{E}_{t,x\sim p_t}\left[\left\|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\right\|_2^2\right] \\ &= \iint \|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\|_2^2\; p_t(x)\,dx\,dt \\ &= \iint \|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\|_2^2\; \left(\int p_t(x\mid z)\,p_{\mathrm{data}}(z)\,dz\right)dx\,dt \\ &= \iiint \|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\|_2^2\; p_t(x\mid z)\,p_{\mathrm{data}}(z)\,dz\,dx\,dt \\ &= \iiint \|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\|_2^2\; p_t(x\mid z)\,p_{\mathrm{data}}(z)\,dt\,dz\,dx \\ &= \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\right\|_2^2\right] \\ &= \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)\right\|_2^2\right] - 2\mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[v^{\theta}(x,t)^{\top}v^{\mathrm{tgt}}(x,t)\right] + \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\mathrm{tgt}}(x,t)\right\|_2^2\right]\end{aligned}$$

And we have:

$$\begin{aligned}\mathcal{L}_{\mathrm{CFM}}(\theta) &= \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)-v(x,t\mid z)\right\|_2^2\right] \\ &= \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)\right\|_2^2\right] - 2\mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[v^{\theta}(x,t)^{\top}v(x,t\mid z)\right] + \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v(x,t\mid z)\right\|_2^2\right]\end{aligned}$$

Since $\mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v(x,t\mid z)\right\|_2^2\right]$ and $\mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\mathrm{tgt}}(x,t)\right\|_2^2\right]$ are both independent of θ, if we find that $\mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[v^{\theta}(x,t)^{\top}v^{\mathrm{tgt}}(x,t)\right] = \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[v^{\theta}(x,t)^{\top}v(x,t\mid z)\right] + \mathrm{constant}$, then the proof is done.

Similiar to how we show $\mathbb{E}_{t,x\sim p_t}\left[\left\|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\right\|_2^2\right] = \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)-v^{\mathrm{tgt}}(x,t)\right\|_2^2\right]$, we find $\mathbb{E}_{t,x\sim p_t}\left[\left\|v^{\theta}(x,t)^{\top}v^{\mathrm{tgt}}(x,t)\right\|\right] = \mathbb{E}_{t,z\sim p_{\mathrm{data}},x\sim p_t(\cdot|z)}\left[\left\|v^{\theta}(x,t)^{\top}v^{\mathrm{tgt}}(x,t)\right\|\right]$. So we have :

$$
\begin{aligned}
\mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[\left\|v^\theta(x,t)^\top v^{\text{tgt}}(x,t)\right\|\right] &= \mathbb{E}_{t,x\sim p_t}\left[\left\|v^\theta(x,t)^\top v^{\text{tgt}}(x,t)\right\|\right] \\
&= \iint v^\theta(x,t)^\top v^{\text{tgt}}(x,t)p_t(x)\,dt\,dx \\
&= \iint v^\theta(x,t)^\top \left(\int v(x,t\mid z)\frac{p_t(x\mid z)p_{\text{data}}(z)}{p_t(x)}\,dz\right)p_t(x)\,dt\,dx \\
&= \iiint v^\theta(x,t)^\top v(x,t\mid z)p_t(x\mid z)p_{\text{data}}(z)\,dz\,dt\,dx \\
&= \iiint v^\theta(x,t)^\top v(x,t\mid z)p_{\text{data}}(z)p_t(x\mid z)\,dt\,dz\,dx \\
&= \mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[v^\theta(x,t)^\top v^{\text{tgt}}(x,t)\right]
\end{aligned}
$$

Amazing! $\mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[v^\theta(x,t)^\top v^{\text{tgt}}(x,t)\right] = \mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[v^\theta(x,t)^\top v(x,t\mid z)\right]$, and we've finished the proof!!!

> Note that the conclusion $\mathcal{L}_{\text{FM}}(\theta) = \mathcal{L}_{\text{CFM}}(\theta) + C$ or $\nabla_\theta\mathcal{L}_{\text{FM}}(\theta) = \nabla_\theta\mathcal{L}_{\text{CFM}}(\theta)$ holds for a more general situation. That is, for a MSE loss function, as long as the error is a linear function of v^{tgt}, this conclusion always holds. For details about this conclusion, refer to **Appendix, II. Conclusion for Linear Error and MSE Loss**.

3.4.2 An Overall Training Procedure

We've showed how to find the conditional velocity, construct the training target, and how to substitute $\mathcal{L}_{\text{FM}}$ with $\mathcal{L}_{\text{CFM}}$. Here is an overall training process to train a nerual network $v^\theta(x,t)$ to approximate $v(x,t)$.

```
# Training Process
Requires:
- neural network: v_theta(x, t)
- training dataset: data
- initial distribution: p_init
- two simple functions: alpha(t) and beta(t)

for each mini-patch in data, do
    get an image z from the dataset                          # z ~ p_data
    sample x_0 from the p_init                               # x_0 ~ p_init
    sampe time t
    x_t = alpha(t) z + beta(t) x_0                           # equal to sampe x_t from p_t( |
z)
    v_cond = d x_t / dt                                      # compute v_cond based on x_t
    L(θ) = ||v_theta(x_t, t) - v_cond||_2^2                  # comupte loss
    update the model parameters θ via gradient descent on L(θ)
end for
```

3.5 Conditional/Guided Generation

3.5.1 From Unconditional to Conditional

So far, the generative model we considered was **unconditional**, e.g. an image model would simply generate some image. However, the task is not merely to generate an arbitrary object, but to generate an object conditioned on some additional information. For example, one might imagine a generative model for images which takes in **a text prompt** y, and then generates an image conditioned on y. For a fixed prompt y, we would thus like to sample from $p_{\text{data}}(\cdot \mid y)$, that is, **the data distribution conditioned on** y. Formally, we think of y to live in a space $\mathcal{Y}$. When y corresponds to a text-prompt, for example, $\mathcal{Y}$ would likely be some continuous space like $\mathbb{R}^{d_y}$. When y corresponds to some discrete class label, $\mathcal{Y}$ would be discrete. For example, for datasets like **MNIST** and **CIFAR10**, the label y are discrete numbers. We will take $\mathcal{Y} = \{0, 1, \dots, 9\}$ for **MNIST** and **CIFAR10**.

To avoid a notation and terminology clash with the use of the word **"conditional"** to refer to conditioning on $z \sim p_{\text{data}}$ (conditional probability path/vector field), we will make use of the term **guided** to refer specifically to conditioning on y. Here, we will refer to e.g., **a guided vector field** $v^{\text{tgt}}(x, t \mid y)$ and **a conditional vector field** $v^{\text{tgt}}(x, t \mid z)$.

So, the key idea of guided generation is to find a guided vector field.

$$\text{Guided Vector Field } v : \mathbb{R}^d \times [0,1] \times \mathcal{Y} \to \mathbb{R}^d, (x, t, y) \to v(x, t \mid y)$$

To approximate $v(x, t \mid y)$, we define a neural netword $v^{\theta}(x, t \mid y)$.

Notice that, the only difference between the guided vector field and the previous vector field is the aditional input $y \sim \mathcal{Y}$. For any fixed label $y \sim \mathbb{R}^{d_y}$, the guided vector field $v(x, t \mid y)$ is equivalent to our previous unguided vector field $v(x, t)$, and we have recovered the unguided generative problem.

> Note that, for guided generation, we say: **sample from** $p_{\text{data}}(\cdot \mid y)$. It seems that y is simply a condition, and it implies that, the procedure is to first sample y from $\mathcal{Y}$, and then sample z from $p_{\text{data}}(\cdot \mid y)$. But, in our image generation case, the fact is that we sample both z and y from p_{data}. This is simply a conceptual difference. This is because, what we have is a **dataset**. Dataset consists of samples from p_{data}. And, in a dataset (with label), what we have is a image-label pair: (z, y).

3.5.2 CFG--A Gaussian Example

3.5.2.1 Expression For Conditional Vector Field

Remember the conditional velocity we mentioned in **3.3.2**? We aim to find an analytical expression for it.

For $z \sim p_{\text{data}}$, $x_0 \sim p_{\text{init}}$ and $x_t \sim p_t(\cdot \mid z)$, we have:

$$v(x_t, t|z) = \frac{dx_t}{dt}, \quad x_t = \alpha_t z + \beta_t x_0$$

$$v\left((\alpha_t z + \beta_t x_0), t \mid z\right) = \dot{\alpha}_t z + \dot{\beta}_t x_0$$

$$x_0 = \frac{x_t - \alpha_t z}{\beta_t}$$

$$v\left((\alpha_t z + \beta_t \cdot \frac{x_t - \alpha_t z}{\beta_t}), t \mid z\right) = \dot{\alpha}_t z + \dot{\beta}_t \cdot \frac{x_t - \alpha_t z}{\beta_t}$$

$$v\left(x_t, t \mid z\right) = \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right)\alpha_t z + \frac{\dot{\beta}_t}{\beta_t} x_t$$

Here, we find the analytical expression for $v(x_t, t \mid z)$.

> Note that, the reason why we need to cancel x_0 is that, here, $v(x_t, t \mid z)$ is a function of (x_t, t, z), instead of a function of x_0. Thus, despite the fact that $v(x_t, t \mid z) = \dot{\alpha}_t z + \dot{\beta}_t x_0$ is simpler than $v\left(x_t, t \mid z\right) = \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right)\alpha_t z + \frac{\dot{\beta}_t}{\beta_t} x_t$, and in fact, when training the model in **3.4** we use $v(x_t, t \mid z) = \dot{\alpha}_t z + \dot{\beta}_t x_0$ to find the conditional velocity. But, the aim of **3.5.2.1** is to find an analytical expression for $v(x_t, t \mid z)$, thus we should not include x_0.

3.5.2.2 Conditional Velocity and Score Function under Gaussian Cases

In generative model, the score function refers to the gradient of the log form of the probability. That is, for a distribution $x \sim p$, $\text{score function: } \nabla_x \log p(x)$. In this section, we will show the relationship between the score function and the conditional velocity under the gaussian case.

> The term $\log p(x)$ is just another form of $p(x)$. Here, $\nabla_x \log p(x)$ points to the direction where the log probability densitity $\log p(x)$ increases with the fast speed. In other words, the score function points to where the log probability densitiy increases the fastest.

Recall the definition of x_t in 3.5.2.1, $x_t = \alpha_t z + \beta_t x_0, \text{where } x_0 \sim \mathcal{N}(0, I_d)$. Thus, $x_t \sim p_t(\cdot \mid z) = N(\alpha_t z, \beta_t^2 I_d)$. According to the pdf of the gaussian distribution, the score function of the distribution $p_t(\cdot \mid z)$ is $\nabla_x \log p_t(x_t \mid z)$. Since $p_t(\cdot \mid z)$ is a gaussian, we conclude that:

$$p_t(x_t \mid z) = c \cdot \exp\left\{-\frac{1}{2}(x_t - \alpha_t z)^T(\beta_t^2 I_d)^{-1}(x_t - \alpha_t z)\right\}$$

$$\log p_t(x|z) = \log(c) - \frac{1}{2}[x_{t1} - \alpha_t z_1,\, x_{t2} - \alpha_t z_2,\, \cdots,\, x_{td} - \alpha_t z_d]\frac{1}{\beta_t^2} I_d \begin{bmatrix} x_{t1} - \alpha_t z_1 \\ \vdots \\ x_{td} - \alpha_t z_d \end{bmatrix}$$

$$= \log(c) - \frac{1}{2\beta_t^2}\sum_{i=1}^{d}(x_{ti} - \alpha_t z_i)^2$$

$$\nabla_{x_t} \log p_t(x_t \mid z) = -\frac{1}{\beta_t^2}(x_{t1} - \alpha_t z_1,\, \cdots,\, x_{td} - \alpha_t z_d)$$

$$= -\frac{x_t - \alpha_t z}{\beta_t^2}$$

$$\alpha_t z = x_t + \beta_t^2 \nabla_{x_t} \log p_t(x_t \mid z)$$

Using this result $\alpha_t z = x_t + \beta_t^2 \nabla_{x_t} \log p_t(x_t \mid z)$ to substitute the $\alpha_t z$ in $v(x_t, t \mid z) = \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right)\alpha_t z + \frac{\dot{\beta}_t}{\beta_t} x_t$, we obtain that:

$$v(x_t, t \mid z) = \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right)\left(x_t + \beta_t^2 \nabla_{x_t} \log p_t(x_t \mid z)\right) + \frac{\dot{\beta}_t}{\beta_t} x_t$$

$$= \frac{\dot{\alpha}_t}{\alpha_t} x_t + \beta_t^2 \left(\frac{\dot{\alpha}_t}{\alpha_t} - \frac{\dot{\beta}_t}{\beta_t}\right)\nabla_{x_t} \log p_t(x_t \mid z)$$

Define:

$$\begin{cases} a_t &= \dfrac{\dot{\alpha}_t}{\alpha_t} \\ b_t &= \beta_t^2\left(\dfrac{\dot{\alpha}_t}{\alpha_t} - \dfrac{\dot{\beta}_t}{\beta_t}\right) \end{cases}$$

We have:

$$v(x_t, t \mid z) = a_t x_t + b_t \nabla_{x_t} \log p_t(x_t|z)$$

This equation $v(x_t, t \mid z) = a_t x_t + b_t \nabla_{x_t} \log p_t(x_t|z)$ is exactly the realtionship between the conditional velocity and the score function.

> Note that, the score function here is to substitute the term $\alpha_t z$. If you simply sonsider a_t, b_t as scaling parameters, the formula $v(x_t, t \mid z) = a_t x_t + b_t \nabla_{x_t} \log p_t(x_t|z)$ means: **the conditional velocity is the addition of the position x_t and the gradient $\nabla_{x_t} \log p_t(x_t \mid z)$, with two scaling parts a_t and b_t.** There's much more interesting things about the score function and the generative model. But here, that' s enough.

3.5.2.3 Score Function and Marginal Velocity

First, we talk about the relationship between two score function: $\nabla_{x_t} p_t(x_t), x_t \sim p_t$ and $\nabla_{x_t} p_t(x_t \mid z), x_t \sim p_t(\cdot \mid z)$.

According to the total probability theroy, we have: $p_t(x_t) = \int p_t(x_t \mid z) p_{\text{data}}(z) dz$. And:

$$\nabla_{x_t} p_t(x_t) = \nabla_{x_t} \int p_t(x_t \mid z) p_{\text{data}}(z) dz$$

4 Mean Flow Model

4.1 Motivation

Flow matching successfully transform a data point from the initial distribution p_{init} into a sample from the target distribution p_{data}. However, during the simulation of ODE, it requires several steps to simulate the final sample. So, can we simply simulate one time to achieve **one-step generation**? Yes, we can. That's the role of mean flow model. Based on flow matching, we use the average velocity instead of the instant velocity to achieve one-step generation.

4.2 Average Velocity

4.2.1 Definition of Average Velocity

For time t and r, suppose that $0 \leq t < r \leq 1$ and the position at time t is x_t (sampled from p_t, i.e., $x_t \sim p_t$), the average velocity during the time t and r is defined as:

$$u(x_t, t, r) \triangleq \frac{1}{r-t} \int_t^r v(x_\tau, \tau)\, d\tau$$

> To emphasize the difference between the instant velocity and the average velocity, through this post, the instant velocity will be denoted as v and the average velocity will be denoted as u.

Note that, the average velocity u is only a function of v, and has no relation with neural network v^θ. That is, $u = f(v) \triangleq \frac{1}{r-t} \int_t^r v\, d\tau$. Conceptually, just as the instantaneous velocity vserves as the ground-truth field in Flow Matching, the average velocity in this formulation provides an underlying ground-truth field for learning. Thus, we will train a model u^θ from scratch, rather than basing u^θ on v^θ.

By definition, the average velocity must satisfy the "consistency" for any $t, r, s \in [0, 1]$:

$$\begin{cases} \lim_{t \to r} u(x_t, t, r) &= v(x_t, t) \\ \\ (r-t) u(x_t, t, r) &= (s-t) u(x_t, t, s) + (r-s) u(x_s, s, r) \end{cases}$$

Our altimate target is to approximate the average velocity with a neural network $u^\theta(x_t, t, r)$. As long as the neural network is accurate enough, we can sample ϵ from p_{init} and use the average velocity $u^\theta(\epsilon, 0, 1)$ to sample from p_{data}. However, directly using $u(x_t, t, r) = \frac{1}{r-t} \int_t^r v(x_\tau, \tau)\, d\tau$ to train the nerual network is intractable, since we have no access to $v(x_t, t)$ and even we have, an integral is eqruired to compute $u(x_t, t, r)$. In the next section, we will show how to construct an optimization target that is amenable to training leveraging the definitional equation of the average velocity.

4.2.2 Mean Flow Identity

To have a formulation amenable to training, we rewrite the average velocity definition as:

$$(r-t)u(x_t,t,r)=\int_t^r v(x_\tau,\tau)\,d\tau$$

Now, differentiating both sides with respect to t. Since r is independent of t:

$$\begin{aligned}\frac{d}{dt}(r-t)u(x_t,t,r)&=\frac{d}{dt}\int_t^r v(x_\tau,\tau)\,d\tau\\ -u(x_t,t,r)+(r-t)\frac{d}{dt}u(x_t,t,r)&=-v(x_t,t)\\ u(x_t,t,r)&=v(x_t,t)+(r-t)\frac{d}{dt}u(x_t,t,r)\end{aligned}$$

We call this equation

$$u(x_t,t,r)=v(x_t,t)+(r-t)\frac{d}{dt}u(x_t,t,r)$$

as the **mean flow identity**, which describes the relationship between the instant velocity v and the average velocity u. Apparently, the **mean flow identity is equivalent to the average velocity definition** (See **Appendix, III. Sufficiency of the MeanFlow Identity** for detials).

4.2.3 JVP to Compute the Derivative

The right side of the **mean flow identity** provides a training target for $u(x_t,t,r)$, which we will leverage to construct a loss function to train a neural network. To serve as a suitable target, we must also further decompose the time derivative term:

$$\frac{d}{dt}u(x_t,t,r)=\frac{\partial u(x_t,t,r)}{\partial x_t}\frac{dx_t}{dt}+\frac{\partial u(x_t,t,r)}{\partial t}\frac{dt}{dt}+\frac{\partial u(x_t,t,r)}{\partial r}\frac{dr}{dt}$$

This equation show that the time partial section can be given by **Jacobian Vector Product (JVP)**.

To be more specifically:

$$u(x_t,t,r):\mathbb{R}^d\times[0,1]\times[0,1]\to\mathbb{R}^d,$$

or $\mathbb{R}^{d+2}\to\mathbb{R}^d$:

$$u(x_t,t,r)=\begin{bmatrix}u_1(x_t,t,r)\\ u_2(x_t,t,r)\\ u_3(x_t,t,r)\\ \vdots\\ u_d(x_t,t,r)\end{bmatrix}_d$$

$\frac{\partial u(x_t,t,r)}{\partial x_t},\frac{\partial u(x_t,t,r)}{\partial t},\frac{\partial u(x_t,t,r)}{\partial r}$ are three **Jacobian Matrixs**:

$$\frac{\partial u(x_t,t,r)}{\partial x_t}=\begin{bmatrix}\frac{\partial u_1}{\partial x_{t1}}&\cdots&\frac{\partial u_1}{\partial x_{td}}\\ \vdots&\ddots&\vdots\\ \frac{\partial u_d}{\partial x_{t1}}&\cdots&\frac{\partial u_d}{\partial x_{td}}\end{bmatrix}_{d\times d},\frac{\partial u(x_t,t,r)}{\partial t}=\begin{bmatrix}\frac{\partial u_1}{\partial t}\\ \vdots\\ \frac{\partial u_d}{\partial t}\end{bmatrix}_{d\times 1},\frac{\partial u(x_t,t,r)}{\partial r}=\begin{bmatrix}\frac{\partial u_1}{\partial r}\\ \vdots\\ \frac{\partial u_d}{\partial r}\end{bmatrix}_{d\times 1}$$

Here,

$$\frac{dx_t}{dt} = v(x_t, t) = \begin{bmatrix} v_1(x_t, t) \\ \vdots \\ v_d(x_t, t) \end{bmatrix}, \frac{dt}{dt} = 1, \frac{dr}{dt} = 0$$

and $\frac{d}{dt}u(x_t, t, r) = \frac{\partial u(x_t, t, r)}{\partial x_t}\frac{dx_t}{dt} + \frac{\partial u(x_t, t, r)}{\partial t}\frac{dt}{dt} + \frac{\partial u(x_t, t, r)}{\partial r}\frac{dr}{dt}$ can be written as:

$$\frac{d}{dt}u(x_t, t, r) = \begin{bmatrix} \frac{\partial u_1}{\partial x_{t1}} & \cdots & \frac{\partial u_1}{\partial x_{td}} \\ \vdots & \ddots & \vdots \\ \frac{\partial u_d}{\partial x_{t1}} & \cdots & \frac{\partial u_d}{\partial x_{td}} \end{bmatrix} \begin{bmatrix} v_1(x_t, t) \\ \vdots \\ v_d(x_t, t) \end{bmatrix} + \begin{bmatrix} \frac{\partial u_1}{\partial t} \\ \vdots \\ \frac{\partial u_d}{\partial t} \end{bmatrix} 1 + \begin{bmatrix} \frac{\partial u_1}{\partial r} \\ \vdots \\ \frac{\partial u_d}{\partial r} \end{bmatrix} 0$$

$$= \overbrace{\underbrace{\begin{bmatrix} \frac{\partial u_1}{\partial x_{t1}} & \cdots & \frac{\partial u_1}{\partial x_{td}} & \frac{\partial u_1}{\partial t} & \frac{\partial u_1}{\partial r} \\ \vdots & \ddots & \vdots & \vdots & \vdots \\ \frac{\partial u_d}{\partial x_{t1}} & \cdots & \frac{\partial u_d}{\partial x_{td}} & \frac{\partial u_d}{\partial t} & \frac{\partial u_d}{\partial r} \end{bmatrix}}_{\text{Jacobian Mateixo of } u: \mathcal{J} \in \mathbb{R}^{d \times (d+2)}} \underbrace{\begin{bmatrix} v_1(x_t, t) \\ \vdots \\ v_d(x_t, t) \\ 1 \\ 0 \end{bmatrix}}_{\text{Tagent Vector: } v \in \mathbb{R}^{d+2}}}^{\text{Jacobian Vector Product (JVP)}}$$

> In modern libraries, **JVP** can be efficiently computed by the **JVP Interface**, such as `torch.func.jvp` in **PyTorch**. Rather than constructing the Jacobian Matrix, modern libraries use the **forward propagation** to compute the JVP.
>
> For simplicity, we write $\frac{d}{dt}u(x_t, t, r)$ as $\frac{\partial u(x_t, t, r)}{\partial x_t}v(x_t, t) + \frac{\partial u(x_t, t, r)}{\partial t}$, but remember in mind that, this is achieved by a single **JVP**, instead of calculating three Jacobian Matrixs and do JVP for three times.

Utilizing the **JVP** result, the mean flow identity can be written as:

$$\begin{aligned} u(x_t, t, r) &= v(x_t, t) + (r - t)\frac{d}{dt}u(x_t, t, r) \\ &= v(x_t, t) + (r - t)\left(\frac{\partial u(x_t, t, r)}{\partial x_t}v(x_t, t) + \frac{\partial u(x_t, t, r)}{\partial t}\right) \end{aligned}$$

From the definition of the average velocity, the mean flow identity, and the JVP method, we simply use the basic calculus. The equation $u(x_t, t, r) = v(x_t, t) + (r - t)\left(\frac{\partial u(x_t, t, r)}{\partial x_t}v(x_t, t) + \frac{\partial u(x_t, t, r)}{\partial t}\right)$ is exactly the analytical expression of the agerage velocity. That's the prototype of the training target. In the next section, we will show how to construct an amenable expression as the training target.

4.3 Training the Model

4.3.1 Constructing the Training Target

Up to now, formulations above are independent of any neural network. We now introduce a model to learn $u(x_t, t, r)$. Formally, we parameterize a neural network $u^\theta(x_t, t, r)$, and encourage it to satisfy the MeanFlow Identity. More specifically, we minimize this objective:

$$\mathcal{L}_{\mathrm{MF}}(\theta) = \mathbb{E}_{t,r,x_t\sim p_t}\left[\left\|u^\theta(x_t,t,r) - \mathrm{sg}\left[u^{\mathrm{tgt}}(x_t,t,r)\right]\right\|_2^2\right]$$

$$\text{where } u^{\mathrm{tgt}}(x_t,t,r) \triangleq v(x_t,t) + (r-t)\left(\frac{\partial u^\theta(x_t,t,r)}{\partial x_t}v(x_t,t) + \frac{\partial u^\theta(x_t,t,r)}{\partial t}\right)$$

Here, the $v(x_t,t)$ is the $v^{\mathrm{tgt}}(x_t,t)$ in flow matching, which implies the "underlying instant velocity vector field". You may noteice that u^{tgt} differs from the mean flow identity, and we add $\mathrm{sg}\left[\cdot\right]$ across the u^{tgt}.

- On the difference between $u^{\mathrm{tgt}}(x_t,t,r)$ and the mean flow identity

  The mean flow identity $u(x_t,t,r) = v(x_t,t) + (r-t)\left(\frac{\partial u(x_t,t,r)}{\partial x_t}v(x_t,t) + \frac{\partial u(x_t,t,r)}{\partial t}\right)$ only reviews the relationship between the instant velocity $v(x_t,t,r)$ and the average velocity $u(x_t,t,r)$, and, you can't find a closed-form expression of $u(x_t,t,r)$, and consequently, using $v(x_t,t) + (r-t)\left(\frac{\partial u(x_t,t,r)}{\partial x_t}v(x_t,t) + \frac{\partial u(x_t,t,r)}{\partial t}\right)$ to compute the training target $u^{\mathrm{tgt}}(x_t,t,r)$ is invalid. Here, we choose to use $\frac{\partial u^\theta(x_t,t,r)}{\partial x_t}$ and $\frac{\partial u^\theta(x_t,t,r)}{\partial t}$ to approximate $\frac{\partial u(x_t,t,r)}{\partial x_t}$ and $\frac{\partial u(x_t,t,r)}{\partial t}$, despite the fact that using the model u^θ itself to construct the training target for u^θ is a little tricky. This approximation is apparently imprecise, but we do this.
- On the $\mathrm{sg}\left[\cdot\right]$ operation

  In the loss function, `stop-gradient`, the $\mathrm{sg}\left[\cdot\right]$ operation, is applied on the training target u^{tgt}. This operation is based on experience instrad of deduction. Here, it eliminates the need for **"double backpropagation"** through the JVP, thereby avoiding higher-order optimization. But, according to Jianlin Su, this operation has something to do with **label leakage** and the ability to achieve **one-step generation**. And, in my perspective, to substitute the marginal velocity with the conditional velocity, we have to add this stop-grad operation (This will be illustrated in **4.3.2**). Anyway, this operation concerns mainly with model training, and has little realtion with the mathmatical deduction. Thus, we simply follow what is said in the Mean Flow Paper, instead of probing into the reason why.

4.3.2 From Loss to "Conditional Loss"

> There's a problem remains. I don't know what's that for. That is, we use $x_t \sim p_t(\cdot \mid z)$, instead $x_t \sim p_t$ as the input of the average velocity $u(x_t,t,r)$. This is not allowed, in my perspective.

We've constructed the training target and the loss function. However, $v(x_t,t)$ is still invalid. Here, we utilize the conclusion in the **3.4.1 Substitute $v^{\mathrm{tgt}}(x,t)$ with $v(x,t\mid z)$: for MSE Loss, if the error is a linear function of $v^{\mathrm{tgt}}(x,t)$, then we can substitute the marginal velocity with the conditional velocity while obtaining the same training results**.

Here:

$$\begin{aligned}\text{error} &= u^\theta(x_t,t,r) - v(x_t,t) - (r-t)\left(\frac{\partial u^\theta(x_t,t,r)}{\partial x_t}v(x_t,t) + \frac{\partial u^\theta(x_t,t,r)}{\partial t}\right)\\ &= \left(u^\theta(x_t,t,r) - (r-t)\frac{\partial u^\theta(x_t,t,r)}{\partial t}\right) - \left(I_d + (r-t)\frac{\partial u^\theta(x_t,t,r)}{\partial x_t}\right)v(x_t,t)\end{aligned}$$

To make it clearer, we define:

$$\begin{cases} b_{\theta,t,r,x_t} & = & \frac{\partial u^\theta(x_t,t,r)}{\partial x_t} v(x_t,t) + \frac{\partial u^\theta(x_t,t,r)}{\partial t} \\ \\ A_{\theta,t,r,x_t} & = & I_d + (r-t)\frac{\partial u^\theta(x_t,t,r)}{\partial x_t} \end{cases}$$

And, the loss function will be rewritten as:

$$\text{error} = b_{\theta,t,r,x_t} - A_{\theta,t,r,x_t} v(x_t,t)$$

$$\text{and the loss function: } \mathcal{L}_{\text{MF}}(\theta) = \mathbb{E}_{t,r,x_t\sim p_t}\left[\|b_{\theta,t,r,x_t} - A_{\theta,t,r,x_t} v(x_t,t)\|_2^2\right]$$

The error is a linear function of the marginal velocity $v(x_t,t)$, and according to the conclusion from **Appendix, II. Conclusion for Linear Error and MSE Loss**, minimizing $\mathcal{L}_{\text{MF}}(\theta) = \mathbb{E}_{t,r,x_t\sim p_t}\left[\|b_{\theta,t,r,x_t} - A_{\theta,t,r,x_t} v(x_t,t)\|_2^2\right]$ is equivalent to minimizing $\mathcal{L}_{\text{CMF}}(\theta) = \mathbb{E}_{t,r,z\sim p_{\text{data}},x_t\sim p_t(\cdot|z)}\left[\|b_{\theta,t,r,x_t} - A_{\theta,t,r,x_t} v(x_t,t\mid z)\|_2^2\right]$. This displacement enable the calculation of the loss.

> Note that, in **Appendix, II. Conclusion for Linear Error and MSE Loss**, we require the matrix A as independent of arguments θ. Here, the matrix A_{θ,t,r,x_t} is not independent from θ. But, as we add the $\text{sg}[\cdot]$ operation on the u^{tgt}, $\dfrac{\partial u^\theta(x_t,t,r)}{\partial x_t}$ will be considered as a constant independent of θ, so does the matrix A_{θ,t,r,x_t} .Thus, in my point of view, to validate the displacement of marginal velocity here, the $\text{sg}[\cdot]$ operation is necessary.

4.3.3 More Training Tricks

- How to sample t and r

  The paper Mean Flow provides two different ways to sample (t,r). The first is to directly sample 2 points from the $\text{Uniform Distribution } U(0,1)$, and then assign the larger one to r, the smaller one to t. The second is to sample ϵ from a $\text{Normal Distribution } N(\mu,\sigma)$, and then use the **logistic function** $f(x) = \dfrac{1}{1+e^{-x}}$ to map the sample ϵ into $[0,1]$. Then, assign the larger one to r and the smaller one to t. The second method is called **"lognorm"**(logistic-norm). Adjusting μ and σ, you can adjust the distribution of t and r.

- Adaptive L2 Loss

  To substitute the marginal velocity with the conditional velocity, the MSE loss is required. But, in practice, with **adapted loss weights** and **MSE Loss**. To be more specifically, MSE loss: $\mathcal{L} = \|\Delta\|_2^2$, and if we adopt the loss form of $\mathcal{L}_{\text{adaptive L2}} = \text{sg}\left[\dfrac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]\cdot\|\Delta\|_2^2$, it's equivalent to the loss $\mathcal{L}_{2\gamma} = \|\Delta\|_2^{2\gamma}$ while keeping the **MSE** form(this guarantees the substitution of marginal velocity with conditional velocity). Here is the proof:

  $$\nabla_\theta \,\text{sg}\left[\frac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]\cdot\|\Delta\|_2^2 = \text{sg}\left[\frac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]\cdot\nabla_\theta\|\Delta\|_2^2$$

  This demonstrates that, minimizing $\mathcal{L}_{\text{adaptive L2}} = \text{sg}\left[\dfrac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]\cdot\|\Delta\|_2^2$ is equivalent to minimizing $\mathcal{L} = \|\Delta\|_2^2$, with the only difference is the **update step**. Here, $\text{sg}\left[\dfrac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]$ serves only as a **scaling part**, influencing only the scale instead of the direction of the gradient. And, that's the reason why we use $\text{sg}\left[\dfrac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]$, the **adapted loss weights**. When error $\|\Delta\|_2^2$ is large, the adpated loss weights $\text{sg}\left[\dfrac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]$ will get small, and vice versa. This weights keep

the gradient in a moderate scale, which helps with training.

We've shown that the loss function $\mathcal{L}_{\text{adaptive L2}} = \text{sg}\left[\frac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right] \cdot \|\Delta\|_2^2$ keep the role of $\mathcal{L} = \|\Delta\|_2^2$, but why this is equivalent to the loss $\mathcal{L}_{2\gamma} = \|\Delta\|_2^{2\gamma}$?

$$\begin{aligned}
\mathcal{L}_{2\gamma} &= \|\Delta\|_2^{2\gamma} \\
&= \left(\|\Delta\|_2^2\right)^{\gamma} \\
\nabla_\theta \mathcal{L}_{2\gamma} &= \gamma\left(\|\Delta\|_2^2\right)^{\gamma-1} \cdot \nabla_\theta \|\Delta\|_2^2 \\
&= \frac{\gamma}{\left(\|\Delta\|_2^2\right)^{1-\gamma}} \cdot \nabla_\theta \|\Delta\|_2^2
\end{aligned}$$

You see, $\frac{\gamma}{\left(\|\Delta\|_2^2\right)^{1-\gamma}}$ and $\text{sg}\left[\frac{1}{(\|\Delta\|_2^2)^{1-\gamma}}\right]$ differs at only a scaling factor γ, and $\nabla_\theta \mathcal{L}_{2\gamma} = \gamma \cdot \nabla_\theta \mathcal{L}_{\text{adaptive L2}}$. Simply neglect this factor γ, they are the same.

> In practice, we will use $\mathcal{L}_{\text{adaptive L2}} = \text{sg}\left[\frac{1}{(\|\Delta\|_2^2 + c)^{1-\gamma}}\right] \cdot \|\Delta\|_2^2$ with a small value c to avoid being devided by zero.

4.3.4 Overall Training/Sampling Procedure

Training Procedure:

```
# Training Process
# fn: Nerual Network to approximate u(x_t, t, r)
for each mini-patch in data, do
    z = get_image()
    t, r = sample_t_r()
    x_0 = sample_from_p_init()

    x_t = x_t = alpha(t) z + beta(t) x_0
    v_cond = d x_t / dt
    u, dudt = jvp(fn, (x_t, t, r), (v, 1, 0))

    u_tgt = v_cond + (r - t) * dudt
    error = u - u_tgt

    loss = adaptive_l2_loss(error)
    update the model parameters θ via gradient descent
end for
```

Sampling Procedure:

```
# Generation/Sample Process
Requires:
- initial distributin: p_init,
- average velocity: u(x, t, r)

sample x_0 from p_init
return x_0 + u(x_0, 0, 1)
```

Appendix

I. Geometric and Analytical Definition of Divergence

$$\lim_{V\to 0}\oint_{\partial V}\mathbf{J}(x,t)d\mathbf{S}_{\mathbf{x}}=\sum_{i=1}^{n}\frac{\partial J_i(x,t)}{\partial x_i}$$

Consider the three dimension case, where $\mathbf{x}=(x,y,z)$. Find the region V as a **cube** centered at $\mathbf{x}$, with edge 2ϵ, and $V=8\epsilon^3$. Call 6 diferent surfaces of the cube as $S_{\pm x}, S_{\pm y}, S_{\pm z}$ where the corresponding outward unit normal vectors are $\pm\hat{x},\pm\hat{y},\pm\hat{z}$.

The quantity outward at the surfaces S_{+x} are:

$$\Phi_{+x}=\int_{y,z\in[-\epsilon,\epsilon]}\mathbf{J}((x+\epsilon,y,z),t)\,d\hat{x}\,dy\,dz$$

or equivalently, for simplicity, written as:

$$\Phi_{+x}=\int_{y,z}\mathbf{J}_{t,x}(x+\epsilon,y,z)\,dy\,dz$$

Similarly,

$$\Phi_{-x}=-\int_{y,z}\mathbf{J}_{t,x}(x-\epsilon,y,z)\,dy\,dz$$

The density out at direction x is:

$$\frac{\Phi_{+x}+\Phi_{-x}}{V}=\frac{1}{8\epsilon^3}\int_{y,z}(\mathbf{J}_{t,x}(x+\epsilon,y,z)-\mathbf{J}_{t,x}(x-\epsilon,y,z))\,dy\,dz$$

Leveraging the definition of differentiable, we have:

$$\mathbf{J}_{t,x}(x\pm\epsilon,y,z)=\mathbf{J}_{t,x}(x,y,z)\pm\epsilon\frac{\partial\mathbf{J}_{\mathbf{t},\mathbf{x}}}{\partial x}+O(\epsilon^2)$$

and,

$$
\begin{aligned}
\frac{\Phi_x}{V} &= \frac{\Phi_{+x} + \Phi_{-x}}{V} \\
&= \frac{1}{4\epsilon^2} \int_{y,z} \frac{\partial \mathbf{J}_{t,x}}{\partial x} \, dy \, dz \\
&= \frac{1}{4\epsilon^2} \frac{\partial \mathbf{J}_{t,x}}{\partial x} \int_{y,z \in [-\epsilon, \epsilon]} dy dz \\
&= \frac{\partial \mathbf{J}_{t,x}}{\partial x}
\end{aligned}
$$

Thus:

$$
\begin{aligned}
\frac{\Phi_{\text{total}}}{V} &= \frac{\Phi_x + \Phi_y + \Phi_z}{V} \\
&= \frac{\partial \mathbf{J}_{t,x}}{\partial x} + \frac{\partial \mathbf{J}_{t,y}}{\partial y} + \frac{\partial \mathbf{J}_{t,z}}{\partial z} \\
&= \sum_{i=0}^{3} \frac{\partial \mathbf{J}_i(x_i, t)}{\partial x_i}
\end{aligned}
$$

II. Conclusion for Linear Error and MSE Loss

For MSE loss, if the error is a linear function of $v^{\text{tgt}}(x, t)$, the conclusion we've deduced in the previous post always holds.

To be more specifically, if matrix A is independent of θ, for linear $\text{error} = Av^{\text{tgt}}(x, t) + b$, MSE loss $\mathcal{L}(\theta) = \mathbb{E}_{t, x \sim p_t} \left[\| Av^{\text{tgt}}(x, t) + b \|_2^2 \right]$ and the conditional MSE loss $\mathcal{L}_{\text{C}}(\theta) = \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\| Av(x, t \mid z) + b \|_2^2 \right]$, the relation $\mathcal{L}(\theta) = \mathcal{L}_{\text{C}}(\theta) + C$ or $\nabla_\theta \mathcal{L}(\theta) = \nabla_\theta \mathcal{L}_{\text{C}}(\theta)$ always holds.

> Note that, we don't care whether the vector b is independent of θ. If b is independent of θ, then the error $Av^{\text{tgt}}(x, t) + b$ or $Av(x, t \mid z) + b$ is apparently independent of θ, thus $\nabla_\theta \mathcal{L}(\theta) = \nabla_\theta \mathcal{L}_{\text{C}}(\theta) = 0$. If the vector b is a function of θ: $b = f(\theta)$, then we hope to prove that $\nabla_\theta \mathcal{L}(\theta) = \nabla_\theta \mathcal{L}_{\text{C}}(\theta)$.

Here is the proof:

$$
\begin{aligned}
\mathcal{L}(\theta) &= \mathbb{E}_{t, x \sim p_t} \left[\left\| Av^{\text{tgt}}(x, t) + b \right\|_2^2 \right] \\
&= \mathbb{E}_{t, x \sim p_t} \left[\left\| Av^{\text{tgt}}(x, t) \right\|_2^2 \right] + 2\mathbb{E}_{t, x \sim p_t} \left[b^\top Av^{\text{tgt}}(x, t) \right] + \mathbb{E}_{t, x \sim p_t} \left[\|b\|_2^2 \right]
\end{aligned}
$$

$$
\text{and}
$$

$$
\begin{aligned}
\mathcal{L}_{\text{C}}(\theta) &= \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\| Av(x, t \mid z) + b \|_2^2 \right] \\
&= \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\| Av(x, t \mid z) \|_2^2 \right] + 2\mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\left\| b^\top Av(x, t \mid z) \right\|_2^2 \right] + \mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\|b\|_2^2 \right]
\end{aligned}
$$

Since A is a matrix independent of θ, $\mathbb{E}_{t, x \sim p_t} \left[\| Av^{\text{tgt}}(x, t) \|_2^2 \right]$ and $\mathbb{E}_{t, z \sim p_{\text{data}}, x \sim p_t(\cdot|z)} \left[\| Av(x, t \mid z) \|_2^2 \right]$ are both independent of θ, thus:

$$\nabla_\theta \mathbb{E}_{t,x\sim p_t}\left[\left\|Av^{\text{tgt}}(x,t)\right\|_2^2\right] = \nabla_\theta \mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[\left\|Av(x,t\mid z)\right\|_2^2\right] = 0$$

For $\mathbb{E}_{t,x\sim p_t}\left[\|b\|_2^2\right]$ and $\mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[\|b\|_2^2\right]$:

$$\begin{aligned}
\mathbb{E}_{t,x\sim p_t}\left[\|b\|_2^2\right] &= \iint \|b\|_2^2\, p_t(x)\, dt\, dx \\
&= \iint \|b\|_2^2 \left(\int p_t(x\mid z) p_{\text{data}}(z)\, dz\right) dt\, dx \\
&= \iiint \|b\|_2^2\, p_t(x\mid z) p_{\text{data}}(z)\, dz\, dt\, dx \\
&= \iiint \|b\|_2^2\, p_t(x\mid z) p_{\text{data}}(z)\, dt\, dz\, dx \\
&= \mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[\|b\|_2^2\right]
\end{aligned}$$

Thus, what's left is to prove:

$$\nabla_\theta \mathbb{E}_{t,x\sim p_t}\left[b^\top A v^{\text{tgt}}(x,t)\right] = \nabla_\theta \mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[\left\|b^\top Av(x,t\mid z)\right\|_2^2\right].$$

Here is the proof:

$$\begin{aligned}
\mathbb{E}_{t,x\sim p_t}\left[b^\top A v^{\text{tgt}}(x,t)\right] &= \iint b^\top A v^{\text{tgt}}(x,t)\, p_t(x)\, dt\, dx \\
&= \iint b^\top A \left(\int v(x,t\mid z)\frac{p_t(x\mid z)p_{\text{data}}(z)}{p_t(x)}\, dz\right) p_t(x)\, dt\, dx \\
&= \iiint b^\top Av(x,t\mid z) p_t(x\mid z) p_{\text{data}}(z)\, dz\, dt\, dx \\
&= \iiint b^\top Av(x,t\mid z) p_t(x\mid z) p_{\text{data}}(z)\, dt\, dz\, dx \\
&= \mathbb{E}_{t,z\sim p_{\text{data}},x\sim p_t(\cdot|z)}\left[\left\|b^\top Av(x,t\mid z)\right\|_2^2\right]
\end{aligned}$$

Thus, $\mathcal{L}(\theta) = \mathcal{L}_{\text{C}}(\theta) + C$ and $\nabla_\theta \mathcal{L}(\theta) = \nabla_\theta \mathcal{L}_{\text{C}}(\theta)$ holds.

III. Sufficiency of the MeanFlow Identity

In our previous post, we show that:

$$u(x_t,t,r) = \frac{1}{r-t}\int_t^r v(x_\tau,\tau)\, d\tau \quad \Rightarrow \quad u(x_t,t,r) = v(x_t,t) + (r-t)\frac{d}{dt}u(x_t,t,r)$$

Here, we aim to show that

$$u(x_t,t,r) = v(x_t,t) + (r-t)\frac{d}{dt}u(x_t,t,r) \quad \Rightarrow \quad u(x_t,t,r) = \frac{1}{r-t}\int_t^r v(x_\tau,\tau)\, d\tau$$

Consider $u(x_t,t,r)$ and $v(x_t,t)$ as functions of t. That is, $u(t) = u(x_t,t,r), v(t) = v(x_t,t)$. Then:

$$u(x_t,t,r) = v(x_t,t) + (r-t)\frac{d}{dt}u(x_t,t,r) \Rightarrow u(t) = v(t) + (r-t)\frac{d}{dt}u(t)$$

Write this ODE as the standard form:

$$\frac{d}{dt}u(t) - \frac{1}{r-t}u(t) = -\frac{1}{r-t}v(t)$$
$$\frac{d}{dt}u(t) + \frac{1}{t-r}u(t) = \frac{1}{t-r}v(t)$$

Then, multiply both sides with another function $g(t)$.

$$g(t)\cdot\frac{d}{dt}u(t) + \frac{g(t)}{t-r}u(t) = \frac{g(t)}{t-r}v(t)$$

View the left side of the equation as:

$$\begin{aligned}\frac{d}{dt}(u(t)\cdot g(t)) &= g(t)\cdot\frac{d}{dt}u(t) + u(t)\cdot\frac{d}{dt}g(t)\\ &= g(t)\cdot\frac{d}{dt}u(t) + \frac{g(t)}{t-r}u(t)\end{aligned}$$

In doing so, we have:

$$\frac{d}{dt}(u(t)\cdot g(t)) = \frac{g(t)}{t-r}v(t)$$

Intrgrate both sides:

$$\frac{d}{d\tau}(u(\tau)\cdot g(\tau)) = \frac{g(\tau)}{\tau-r}v(\tau)$$

$$d(u(\tau)\cdot g(\tau)) = \frac{g(\tau)}{\tau-r}v(\tau)\,d\tau$$

$$\int_t^r d(u(\tau)\cdot g(\tau)) = \int_t^r \frac{g(\tau)}{\tau-r}v(\tau)\,d\tau$$

$$u(r)\cdot g(r) - u(t)\cdot g(t) = \int_t^r \frac{g(\tau)}{\tau-r}v(\tau)\,d\tau$$

Then, we need to find $g(t)$. To do so, we have:

$$\begin{aligned}\frac{d}{dt}(u(t)\cdot g(t)) &= g(t)\cdot\frac{d}{dt}u(t) + u(t)\cdot\frac{d}{dt}g(t)\\ &= g(t)\cdot\frac{d}{dt}u(t) + \frac{g(t)}{t-r}u(t)\end{aligned}$$

$$\Downarrow$$

$$\frac{dg(t)}{dt} = \frac{g(t)}{t-r}$$

Thus:

$$\frac{1}{g(t)}dg(t) = \frac{1}{t-r}dt$$

$$\ln\left(g(t)\right) = \ln|t-r| + C$$

$$\begin{aligned}g(t) &= A\cdot\exp\left(\ln|t-r|\right)\\ &= A(r-t), \quad \text{where} A = e^C > 0.\end{aligned}$$

Substitute $g(t) = A(r-t)$ in the equation $u(r) \cdot g(r) - u(t) \cdot g(t) = \int_t^r \frac{g(\tau)}{\tau - r} v(\tau)\, d\tau$, we obtain:

$$u(r) \cdot A(r-r) - u(t) \cdot A(r-t) = \int_t^r \frac{A(r-\tau)}{\tau - r} v(\tau)\, d\tau$$

$$u(t)(r-t) = \int_t^r \frac{A(\tau-r)}{\tau - r} v(\tau)\, d\tau$$

$$u(t) = \frac{1}{r-t} \int_t^r v(\tau)\, d\tau$$

Thus:

$$u(x_t, t, r) = v(x_t, t) + (r-t)\frac{d}{dt} u(x_t, t, r) \quad \Rightarrow \quad u(x_t, t, r) = \frac{1}{r-t} \int_t^r v(x_\tau, \tau)\, d\tau$$